

Программирование на Python

Тема 1. Лекция 1

Введение в Python и библиотеки для анализа данных

Юрий Саночкин

ysanochkin@hse.ru

НИУ ВШЭ, 2025

Что такое Python?

- Язык программирования
- Для чего он используется? Какие задачи решаются с помощью Python?
- Относительно медленный язык, в сравнении с С-подобными
- Не подходит для высоконагруженных систем
- Не очень удобен для разработки, хотя используется
- Самый распространенный язык для анализа данных, аналитики, машинного обучения

Особенности Python

- Что такое строго типизированный и нестрого типизированный язык программирования?
- Типы данных есть в любом языке, но в Python они задаются неявно

Код в Java (строгая типизация)

```
int a = 10;  
  
int b = a + 1;
```

Код в Python (нестрогая типизация)

```
a = 10  
  
b = a + 1
```

Особенности Python

- Важны ли отступы в языках Java, C#?
- Во многих языках отступы используются для красоты, а для навигации между блоками используются точки с запятой, скобки и другие символы
- В Python последовательные смысловые блоки обязаны иметь одинаковый отступ

Элементарные типы данных

- Целые числа (int)

```
a = 10  
b = -2456
```

- Нецелые числа (float)

```
a = 5.5  
b = -3.789
```

- Строки (string)

```
a = "Hello"  
b = 'world'
```

- Бинарные (bool)

```
a = True  
b = False
```

Ввод и вывод

```
a = input()
b = int(input())
```

```
print("Hello", "my")
print("dear")
print("world")
```

```
print("Hello", "my", sep = ", ", end = " ")
print("dear")
print("world")
```

```
Hello my
dear
world
Hello, my dear
world
```

Условные конструкции

```
if a > 0:  
    print("Positive number")  
elif a < 0:  
    print("Negative number")  
else:  
    print("Zero")
```

Циклы

- Какие вы знаете циклы?
- В Python есть только 2 цикла: `for`, `while`
- Во многих языках даже эти циклы можно реализовать друг через друга, но в Python — они имеют немного разную природу.

Цикл for

```
print("school age:")  
for i in range(6, 19):  
    print(i)
```

Что делает команда range ?

school age:

6

7

8

9

10

11

12

13

14

15

16

17

18

Цикл while

```
a = 5
while a > 0:
    print("Another iteration")
```

В чем тут проблема?

[illegible]

Цикл while

```
a = 5
while a > 0:
    a -= 1
    print("Another iteration")
```

Another iteration
Another iteration
Another iteration
Another iteration
Another iteration

```
print("waiting for negative:")
while True:
    a = int(input())
    if a >= 0:
        print("Not negative")
    else:
        print("We won!")
        break
```

waiting for negative:
10
Not negative
4
Not negative
3
Not negative
-7
We won!

Списки

- Чем отличаются списки от массивов?
- В Python — ничем, это одно и то же

```
A = [1, 2, 3, 10]
B = []
C = [1, "Hello", 3.5, False, [1, 2, 3]]
```

Срезы

- Индексация и срезы применяется к любой последовательности
- Какие бывают последовательности (коллекции)?
- Существует очень много разных последовательностей. Из уже известных нам: строки, списки. Также обсудим кортежи. Но вообще, их гораздо больше.

Срезы

- Что запишется в В в каждом из этих случаев?

```
A = [1, 2, 3, 4, 5, 6]
```

```
B = A[1]
```

```
B = A[-3]
```

```
B = A[1:4]
```

```
B = A[3:]
```

```
B = A[2:5:2]
```

```
B = A[4:2:-1]
```

```
B = A[::-1]
```

Функции и методы

- Функции и методы звучат очень похоже и часто в речи в бытовом смысле, наверное, взаимозаменяемы.
- В чем же разница между ними?
- Методы применяются к конкретному объекту (к конкретной переменной, если проще, хотя с ООП вы вроде знакомы 😊), функции — без привязки к объекту.
- У каждого типа данных методы свои. Функции не зависят от этого и имеют только входные параметры.

Основные методы для строк

- `find` — индекс первого вхождения подстроки в строку
`S.find('a')`
- `rfind` — индекс последнего вхождения подстроки в строку
`S.rfind('a')`
- `replace` — заменяет все вхождения одной подстроки на другую
`S.replace('a', 'AA')`
- `count` — подсчитывает количество вхождений подстроки в строку
`S.count('a')`
- `split` — разбивает строку по разделителю в массив
`S.split(';')`
- `join` — объединяет элементы последовательности по разделителю
`' '.join(A)`

Основные методы для списков

- `append` — вставляет элемент в конец списка
`A.append(10)`
- `pop` — удаляет элемент по индексу
`A.pop(3)`
- `index` — индекс первого вхождения элемента в список
`A.index(10)`
- `count` — подсчитывает количество вхождений элемента в список
`A.count(10)`

Некоторые встроенные функции

- `len` — длина последовательности
- `min` — ищет минимальный элемент последовательности
- `max` — ищет максимальный элемент последовательности
- `sum` — находит сумму последовательности
- `map` — применяет к каждому элементу последовательности указанную функцию
- ...

Пользовательские функции

- Функцию можно создать и самому
- А зачем?
- Почему бы не писать код одним блоком?
- Читаемость кода
- Деление кода на независимые смысловые блоки
- Многократное воспроизведение определенного функционала
- Расширяемость кода
- и многое другое...

Пользовательские функции

- То, что, можно сказать, понадобится нам больше всего в анализе данных... 😊

```
def factorial(n):  
    if n == 0:  
        return 1
```

```
    prod = 1
```

```
    for i in range(1, n + 1):  
        prod *= i
```

```
    return prod
```

Что это?

```
def binom_n_k(n, k):  
    return factorial(n) // (factorial(n-k) * factorial(k))
```

Последний штрих

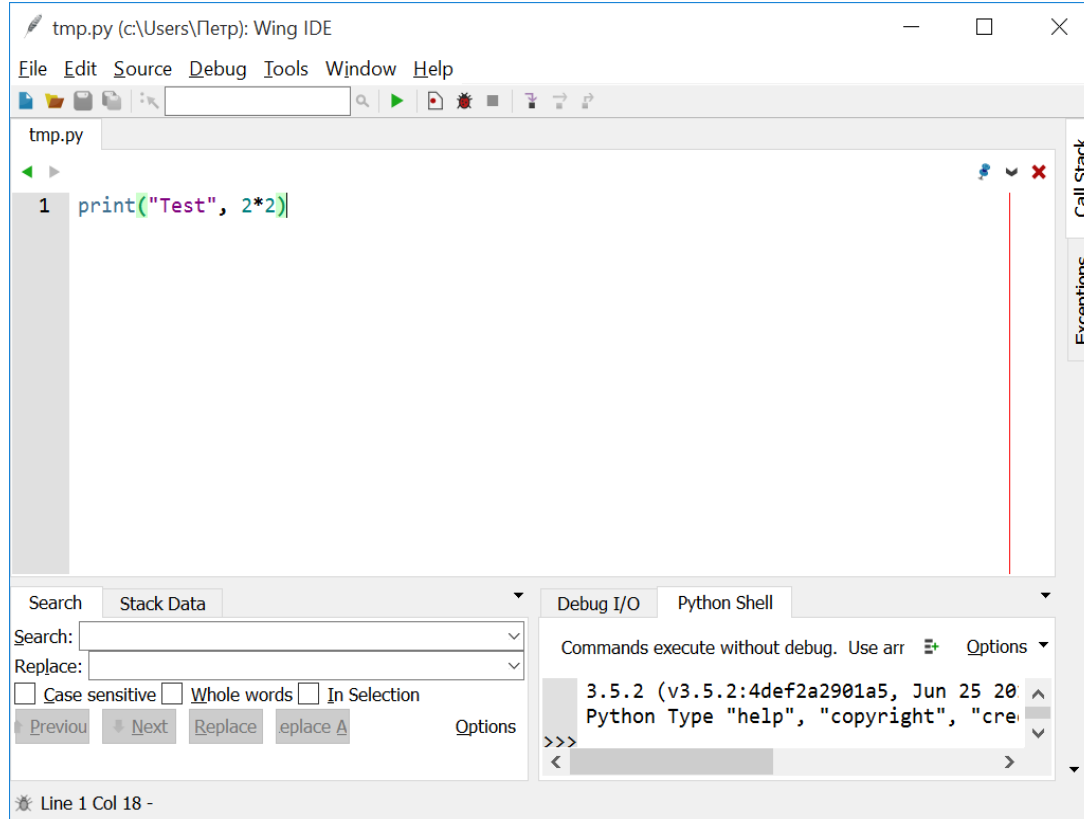
- А что такое кортежи?
- Чем они отличаются от списков?
- По-простому, такие же списки, только неизменяемые

```
a = (1, 2, 3)
```

```
a = 1, 2, 3
```

Среда разработки

- Wing IDE ?



- PyCharm ?

